

Windows OS 기반 대화형 AI 애플리케이션에 대한
포렌식 아티팩트 분석
여준석 · 김채민 · 김동현

사단법인 한국포렌식학회

KOREAN INSTITUTE OF FORENSIC SCIENCES

Windows OS 기반 대화형 AI 애플리케이션에 대한 포렌식 아티팩트 분석

여준석*, 김채민, 김동현**

국문초록

최근 생성형 인공지능의 발전은 혁신을 이끌고 있지만, 프라이버시 침해와 데이터 오용 위험도 증가하고 있다. 본 연구는 5개의 Windows 기반 대화형 AI 애플리케이션을 대상으로 디지털 포렌식 분석을 수행하여 사용자 행위로 생성된 애플리케이션의 아티팩트를 체계적으로 정리하고, 가상 시나리오를 통해 민감 정보가 유출될 수 있는 보안 사고 조사에서의 활용 가능성을 제시하였다. 이를 통해 민감 정보 유출 방지와 악용 사례 식별을 위한 실질적인 방안을 제공하고자 한다. 본 연구의 결과가 악의적으로 콘텐츠를 생성하는 악용자의 컴퓨터를 조사하는 데 활용되고, 민감한 정보 유출을 미연에 방지하는 데 기여할 수 있기를 기대한다.

주제어: 디지털 포렌식, 애플리케이션 포렌식, 아티팩트 분석, 대화형 AI

I. 서론

최근 생성형 인공지능(Generative AI)의 발전은 의료, 예술, 금융, 사이버 보안 등 다양한 분야에서 혁신을 이끌며 개인과 기업 모두에게 큰 가치를 제공하고 있다. 특히, 대규모 언어 모델(Large Language Model)은 방대한 데이터 학습을 기반으로 사용자에게 최적화된 응답을 제공하며, 아이디어 획득, 프로그래밍 등 다양한 작업을 수행할 수 있는 도구로 자리 잡았다. 이러한 변화는 사용자의 생산성을 극대화하는 동시에, 새로운 비즈니스 모델을 창출하고 있다.

그러나 대규모 언어 모델 기반 AI 서비스의 확산은 프라이버시 침해와 사

* 제1저자: 육군사관학교 컴퓨터과학과 학사과정, hey_mr_jun@kma.ac.kr

** 교신저자: 육군사관학교 컴퓨터과학과 조교수, winer1492@kma.ac.kr

용자 데이터 오용 문제를 야기할 가능성을 내포하고 있다. 사용자가 입력한 데이터가 AI 모델의 학습 데이터로 재활용될 경우, 기업 및 국가의 민감한 정보가 유출될 위험이 있으며, 이는 심각한 경제적, 사회적 손실로 이어질 수 있다. 또한, 대규모 언어 모델은 매번 다른 텍스트를 생성할 수 있는 특성을 보여, 스팸 메일, 가짜 뉴스, 댓글 조작 등 악의적인 목적으로 악용될 가능성도 존재한다.

이와 같은 문제를 해결하기 위해 대규모 언어 모델 서비스에 대한 디지털 포렌식 연구의 필요성이 대두되고 있다. 특히, 웹 서비스, 웹 브라우저 확장 프로그램, 설치형 프로그램 등 다양한 형태로 제공되는 대규모 언어 모델 서비스는 각각의 특성에 따라 포렌식 분석 방법이 달라질 수 있다. 따라서, 각 서비스 형태에 적합한 포렌식 분석 방법론을 개발하고, 이를 통해 민감 정보 유출 및 악용 사례에 대해 포렌식 분석할 방법을 개발하는 것은 중요하다.

본 연구는 Windows 기반 대화형 AI 애플리케이션(application)을 대상으로 디지털 포렌식 분석을 수행한다. 총 5가지의 애플리케이션을 선정하여, 애플리케이션 실행과 종료, 대화 기록, 업로드한 파일 등과 같이 사용자와 관련된 아티팩트가 애플리케이션에서 기록되는지 확인한다. 또한, 민감 정보가 유출될 수 있는 보안 사고를 가정한 가상 시나리오를 통해 포렌식 조사에서 유의미하게 사용될 수 있음을 보인다. 본 연구의 기여는 다음과 같다.

- Windows 기반 대화형 AI 애플리케이션의 포렌식 분석 필요성 강조
- 총 5개의 애플리케이션을 선정 후 체계적으로 아티팩트 정리
- 시나리오를 통해 AI 애플리케이션 오용으로 인한 보안 사고 조사에 유용함을 보임

II. 관련 연구

AI 서비스는 ChatGPT, Gemini, Claude 등 다양하며, 제공되는 형태도 모바일 웹, 데스크톱 웹 등으로 나뉜다. 이러한 서비스와 애플리케이션에 대한 분석을 선행 연구 3개를 확인할 수 있었다. 선행 연구들은 각 제공 형태에 따른 AI 서비스에 대해 포렌식 분석을 수행했으며, 대부분 ChatGPT와 같이 잘 알려진 서비스에서 제공하는 애플리케이션을 분석하였다.

조경석 등(Cho et al., 2024)는 웹 서비스로 제공되는 대화형 AI인 ChatGPT, Gemini, Copilot, Claude를 대상으로 포렌식 분석을 수행했다. 이 연구

에서는 설치형 애플리케이션은 포렌식 분석 대상으로 고려하지 않았으며, 웹 기반 API, 웹 캐시, 메모리 덤프 분석을 통해 대화 기록, 검색 기록, 사용자 입력 데이터를 식별하고자 했다. 또한, 데이터 수집 → 데이터 분석 → 서비스별 특화된 절차 → 결과 보고서 작성의 4단계로 이루어진 포렌식 프레임워크를 제안하였다. 하지만 웹 서비스 형태로 제공되는 대화형 AI만을 대상으로 한 연구로, 그 범위가 제한적이다.

Walker 등(Walker et al., 2024)는 마이크로소프트(Microsoft)에서 제공하는 다중 에이전트 기반 대규모 언어 모델 플랫폼(platform)인 AutoGen에 대해서 포렌식 분석을 수행하였고, AutoGen 실행으로 기록되는 메모리, 디스크, 네트워크 아티팩트를 식별하였으며, 디스크와 네트워크에서 유의미한 흔적을 발견할 수 있었다. 이 연구는 인간 사용자와 AutoGen 에이전트(agent)가 생성한 아티팩트를 구분하는 데 초점을 두고 분석하였지만, 사용자와 에이전트를 명확히 구분할 수 있는 근거를 발견하지는 못했다. 그럼에도 불구하고, AutoGen에 대한 포렌식 방법론을 제안했다는 점에서 의의가 있다.

Dragonas 등(Dragonas et al., 2024)는 안드로이드(Android)와 iOS에서 동작하는 ChatGPT 애플리케이션을 포렌식 분석하였다. 이 연구는 저장장치에 기록되는 아티팩트와 내보내기 기능으로 확인할 수 있는 클라우드 데이터를 분석하였으며, 이를 통해 대화 기록과 애플리케이션 사용 흔적 등을 식별할 수 있음을 주장하였다. 이 연구는 ChatGPT 모바일 애플리케이션에 대한 최초의 연구라는 점에서 의의가 있다.

AI 서비스가 널리 활용되고, AI 모델 개발, 활용법에 관한 문헌 연구는 많지만, AI 서비스와 애플리케이션에 대한 포렌식 분석 연구는 부족한 실정이다. 특히, AI 모델을 활용하는 서드파티(third-party) 애플리케이션이 증가함에 따라, 사용자의 실수나 악의적인 사용으로 인해 AI 서비스와 애플리케이션이 문제의 중심이 될 가능성이 높아지고 있다. 이러한 상황에서, 서드파티 AI 애플리케이션에 대한 포렌식 분석 연구가 필요하다. 본 연구는 서드파티 AI 애플리케이션을 대상으로 포렌식 분석을 수행한다는 점에서 선행 연구들과 차별화된 기여를 제공한다.

III. 대화형 AI 애플리케이션 포렌식 분석

1. 분석 대상 선정

본 연구에서는 Windows에서 설치되어 동작하는 대화형 AI 애플리케이션에서 기록되는 아티팩트를 분석한다. 총 5가지 분석 대상을 선정하였고, 이들은 Monica*, Msty**, Jan***, LM Studio****, Sanctum*****이다. 이 분석 대상들을 선정하기 위해, AI 서비스에 대한 정보를 제공하는 Huggingface AI*에서 Windows 기반의 대화형 AI 애플리케이션을 조사하였다. Monica의 경우 데스크톱 구동을 지원하는 대중적으로 많이 알려진 AI 애플리케이션이기에 따로 분석 대상으로 선정하였다. 그리고 Msty, Jan, LM Studio, Sanctum은 Huggingface AI에서 공식적으로 인증 및 지원 중인 Text Generation Local Apps 중 데스크톱 구동도 가능한 모델들이었기에 분석 대상으로 선정하였다. 각 애플리케이션의 개발사, 버전은 <표 1>과 같다.

<표 1> 분석 대상 애플리케이션 정보

서비스 이름	개발사(개발자)	버전
Monica	BUTTERFLY EFFECT PTE. LTD.	2.7.1
LM Studio	lmstudio-ai	0.3.5
Msty	CloudStack LLC.	0.3.14
Jan	Homebrew Computer Company	0.5.9
Sanctum	Sanctum Labs	1.8.2

2. 분석 방법론

* BUTTERFLY EFFECT PTE. LTD., Monica. <https://monica.im/desktop> (검색일: 2024년 11월 30일)

** CloudStack LLC. Msty. <https://msty.app> (검색일: 2024년 11월 30일)

*** Homebrew Computer Company. Jan. <https://jan.ai> (검색일: 2024년 11월 30일)

**** lmstudio-ai. LM Studio. <https://lmstudio.ai> (검색일: 2024년 11월 30일)

***** Sanctum Labs. Sanctum. <https://sanctum.ai> (검색일: 2024년 11월 30일)

* “Hugging Face Local Apps and Hardware”. Hugging Face. <https://huggingface.co/settings/local-apps> (검색일: 2024년 11월 30일)

1) 실험 및 분석 환경

실험 및 분석은 외부와 네트워크가 연결된 Windows 11 운영체제를 사용한 컴퓨터에서 수행되었다. 실험을 위한 분석 환경 및 분석 도구는 <표 2>에 정리하였다. Sanctum 애플리케이션을 제외한 나머지 분석 대상들은 크로미엄(Chromium) 기반으로 동작했기 때문에 ChromeCacheView 도구를 활용했고, 업로드한 파일의 캐시(cache)를 확인하기 위해 HxD를 사용했다. 마지막으로, 애플리케이션 사용 후 변경되는 파일 기반 아티팩트를 확인하기 위해서 Process Monitor를 활용했다.

<표 2> 실험 분석 환경 및 도구

항목	세부 내용
운영 체제	Windows 11 Home 64-bit (Version 23H2 / 22631.4460)
저장 공간	SSD 1TB
네트워크 환경	개방형 네트워크
분석 도구	Chromecacheview, HxD, Process Monitor

2) 대규모 언어 모델 질의문 양식

분석 대상으로 선정된 애플리케이션 분석에서 일관성 있는 결과를 얻기 위해, 다음과 같은 3가지의 통일된 양식으로 각 대규모 언어 모델에 질문하였다.

1. 대규모 언어 모델의 성능 지표에는 어떤 것이 있는지 구체적으로 알려줘.
2. FIFO 알고리즘을 python으로 구현해서 코드를 나에게 보여주고, 설명해 줘.
3. 이 python 코드 파일의 동작 방식과 알고리즘을 분석해서 나에게 설명해 줘.
 - 3.1. [그림 1]과 같은 'lets_recover.py' 파일 업로드.
 - 3.2. 파일 업로드 기능 비활성화 시, 코드를 텍스트로 첨부.

```
# 복호화 키
v6 = [0xDE, 0xAD, 0xBE, 0xEF]
# 파일 열기
with open('encrypted', 'rb') as encrypted_file, open('classified.png', 'wb') as
recovered_file:

    ...

    recovered_file.write(decrypted_data)
print("복호화 완료. classified.png 파일이 복원되었습니다.")
```

[그림 1] lets_recover.py

3. 클라우드 기반 애플리케이션 아티팩트 분석

1) Monica

Monica 애플리케이션은 사용자가 반드시 계정을 등록하여야만 사용할 수 있으며, 로그아웃 상태에서는 사용할 수 없다. <표 3>과 같이 포렌식적으로 유의미한 데이터는 main.log 파일에 기록됨을 확인했다. config.json 파일의 경우 암호화된 상태로 저장되어 있으며, 그 외 데이터는 암호화 없이 저장되어 있다.

- 아티팩트 기록 경로: %APPDATA%\Roaming\Monica\

<표 3> Monica 아티팩트 세부 경로 및 항목

아티팩트 세부 경로	아티팩트 항목
%APPDATA%\Monica\logs\main.log	<u>program running info log, file upload log</u> , ChatLRUCache info, <u>creation time</u>

Monica의 경우 사용 방법에 따라 기록되는 아티팩트가 달랐다. ChatGPT 4 모델을 사용하는 경우(1), 상세한 로그가 남지 않았지만, Monica 모델을 사용하는 경우(2), 대화 내용, 파일 업로드 정보, 대화 생성 시각과 같은 상세 대화 로그가 남았다.

(1) ChatGPT 4 모델을 사용한 경우

[그림 2]와 같이, main.log에는 오로지 “ChatLRUCache” 값이 업데이트되었다는 로그만 반복해서 기록된다. 이때, ChatLRUCache는 Chat Least Recently

Used Cache로, chat cache 데이터에 공간이 부족할 때 가장 오랫동안 사용하지 않은 항목을 제거하고 새로운 항목을 배치하는 형식으로 동작하는 캐시이다. 기록되는 로그 생성 시각은 설치된 컴퓨터에 설정된 시간대를 따른다.

```
[2024-11-26 12:34:46.207] [info] [ChatLRUCache] update
conv:3827da26-75d4-48d9-8eb1-548e2a1aa7c9
...
[2024-11-26 12:35:43.473] [info] [ChatLRUCache] update
conv:3827da26-75d4-48d9-8eb1-548e2a1aa7c9
```

[그림 2] Monica에서 외부 모델 사용 시 main.log에 기록된 로그

로컬에서는 캐시 정보와 관련하여 포렌식적으로 유의미한 추가 로그 및 아티팩트가 형성이 안 되고 있지만, 웹 애플리케이션 URL과 연동할 경우 유일하게 웹상에서 복구가 가능함을 분석하였다. 단, URL로 복구 시 해당 채팅을 기록하였던 계정으로 로그인되어 있어야 하므로, 사용자 계정 정보를 획득하지 못한 경우 복구가 어려울 수 있다. Monica 웹 URL 구조는 다음과 같다.

- URL: `https://monica.im/home/chat/{model name}/{model name}?convId=conv%3A{updated conv value}`
- URL 예시 : `https://monica.im/home/chat/GPT-4o_mini/gpt_4o_mini_chat?convId=conv%3A3827da26-75d4-48d9-8eb1-548e2a1aa7c9`

(2) Monica 모델을 사용한 경우

Monica 모델을 이용하여 질의할 경우, 질의한 흔적은 JSON의 형식으로 main.log에 기록된다. 첨부한 파일은 “data” 하위의 “fileinfos”에 메타데이터가 기록되고 있으며, “fileUrl”의 값을 그대로 웹 브라우저에 입력하여 검색할 경우, 해당 파일을 복구할 수 있다. 다만, ‘Expires={Unix TimeStamp}’ 형태로 저장 기한이 정해져 있으므로, 해당 기한 내에 접근하여야만 포렌식적으로 유의미한 복구가 가능하다. 추가 질문 관련 main.log 내용 및 세부 분석 내용을 [그림 3] 및 <표 4>와 같이 정리하였다.


```
conversationId: 'convra5e8b361-157a-4948-b095-e640e8f0e48e',
itemId: 'msg:clfc2b9d3-ab15-40f9-b3af-07f9aa023bfd',
itemType: 'question',
summary: '이 그림에 대해 어떻게 생각해?',
parentItemId: 'msg:5f188280-b8ac-4080-9eba-5c539c226f10',
data: {
  type: 'file_with_text',
  content: '이 그림에 대해 어떻게 생각해?',
  fileInfo: {
    useFullText: true,
    fileExt: 'jpeg',
    fileName: 'H3IND33R logo.webp',
    fileSize: 687462,
    fileUrl: 'https://d41tchsnpgdne.cloudfront.net/user_upload_by_module/chat_bot/files/47964923/XZHkYtURsQgV/LMF.webp?Expires=1730383408&Signature=Z4fYUtzONsZV8Czko3ZnuoaYhye95YVY55uUBV0UEKdqo2TbJ5ioPoQ10TcUJKQppintucW6YUafgycvaUmnvYPKawYtpfOVQseyNIM3v~LhhAOckAzHdb3H6DfysPhgtrf3-Bw45iMsYS2fwsMvst1BL6UWPNPQabZ2Q9yKisRjmwf5y-kww5D8VDABRbIPagMkIWBIKbe~OQe47fR4JgWGLzCA~bd0c4gG4sisiu9KFT9zdo2e7~ZcQ1DLZnA1F4PCr073vQOEyBIRTwes5fkoEIE4ne70s0Pms3U7d5EbadHMTdGCFRti8UE68Ak~c9PairSQ__8Key-Pair-Id=K3USGZIKWMDCC5X',
    fileId: 'a5e6765bfede4b3aae92105ecf2cb32b',
    fileChunks: 1,
    fileTokens: 0
  }
},
maxToken: 0
}
}
}
createdAt: 1729173328650,
hasMore: false,
updatedAt: 1729173328650,
cacheVersion: 10000000,
chatBotUid: 'monica',
title: 'AI 어시스턴트 소개'
}
```

[그림 3] Monica의 자체 모델 사용 시 main.log에 기록된 로그

<표 4> main.log에 기록되는 로그 항목 및 세부 내용

항목	세부 내용
conversationID	현재 대화창의 고유 식별 ID
itemID	메시지 고유 식별 ID
summary	텍스트 요약
itemType	메시지 종류 type: question, reply
parentitemID	참조한 itemID
data	데이터 세부 사항: useFullText, fileExt, fileName, fileSize, fileURL, fileId, fileChunks, fileTokens
createdAt	<u>아티팩트 생성 시각(Unix Timestamp)</u>
updatedAt	<u>아티팩트 최신 업데이트 시각(수정, 삭제 등)</u>
cacheVersion	chat Cache 버전
chatBotUid	채팅 중인 봇 식별 UID
title	채팅방 제목

2) Msty

Msty 애플리케이션은 외부 대규모 언어 모델 API 공유 기능은 물론, 로컬 gguf 파일 다운로드 및 설치를 통해서도 사용할 수 있다. 포렌식적으로 유의미한 데이터는 <표 5>의 구조와 같이 저장되는 것으로 확인되었다. %APPDATA%\Msty\msty.db를 제외한 아티팩트에 기록되는 모든 시간대는 설치된 컴퓨터에 설정된 시간대를 따른다.

- 아티팩트 기록 경로: %APPDATA%\Msty

<표 5> Msty 아티팩트 세부 경로 및 항목

아티팩트 세부 경로	항목
%APPDATA%\Msty\attachments	<u>copies of uploaded files</u>
%APPDATA%\Msty\Cache\Cache_Data	local gguf file metadata
%APPDATA%\Msty\msty.db	<u>conversation log</u> , <u>API Key</u>
%APPDATA%\Msty\logs\app.log	<u>app running log</u> , loaded model info, error log

%APPDATA%\Msty\attachments. 첨부한 문서 혹은 파일을 복사하여 자동으로 저장하는 폴더이다. 업로드된 파일의 이름은 {created_time(Unix Times tamp)}-{original file name}.{extension} 형식으로 저장된다. 따라서 사용자가 업로드한 파일을 복구하고자 할 경우, 해당 폴더에 진입하면 된다. [그림 4]는 저장된 파일이 담긴 attachments 폴더의 사진이다.



[그림 4] Msty의 attachments 폴더에 저장된 업로드 파일 복사본

%APPDATA%\Msty\Cache\Cache_Data. 챗봇과의 대화를 위해 로드한 로컬 gguf 모델 파일에 관한 정보가 담겨 있다. ChromeCacheView를 이용하여

해당 폴더를 분석하면, filename이 “tags”인 콘텐츠가 있으며, 해당 URL로 접속하면 [그림 5]와 같은 model 정보가 표시된다. 모델 생성 및 수정 시각은 설치된 컴퓨터에 설정된 시간대를 따른다.

```
"models": [
  {
    "name": "deepseek-coder-v2:latest",
    "model": "deepseek-coder-v2:latest",
    "modified_at": "2024-11-07T18:18:37.5910979+09:00",
    "size": 8905126121,
    "digest": "63fb193b3a9b4322a18e8c6b250ca2e70a5ff531e962dbf95ba089b2566f2fa5",
    "details": {
      "parent_model": "",
      "format": "gguf",
      "family": "deepseek2",
      "families": {
        "deepseek2"
      },
      "parameter_size": "15.7B",
      "quantization_level": "Q4_0"
    }
  }
]
```

[그림 5] Msty의 Cache_Data에 기록된 로컬 모델 파일 정보

%APPDATA%\Msty\msty.db. 각종 채팅방 정보와 세부 채팅 내용까지 모두 기록하여 통합적으로 저장하는 SQLite 데이터베이스 파일이다. chat_messages 테이블을 통해 대화 내용을 상세히 기록해 둔다. 이 외에도 api_keys 테이블에 사용자가 사용하고 있는 api 키값이 암호화 없이 그대로 기록된 것을 확인할 수 있다. msty.db의 구조는 <표 6>와 같다. 또한 테이블 중 api_keys에 들어가면, [그림 6]과같이 OpenAI 대화를 위해 저장해 두었던 API 키가 그대로 노출되어 보이는 것을 확인할 수 있다.

데이터(T): api_keys							
	id	name	key	provider	created_at	key_hint	mo
1	01JC2WFWCDKEW9Z5G7JQT0YSRZ	openai	sk-proj-nOQLVja---	openai	2024-11-07 08:40:09	sk-...A2IA	{'purpose':...

[그림 6] api_keys 테이블에 기록된 사용자 OpenAI API 키

<표 6> msty.db의 세부 구조

테이블명	설명	칼럼명
api_keys	사용한 API 키 목록	id, name, key , provider , created_at , key_hint, models, extras, save_in_keychain
chat_messages	챗봇 대화 로그	id , chat_id , text , config, role , sync_buddy_id, refinement_prompt, parent_id, branch_parent_id, model_name, attachments , extras, refinement_active_id, created_at , knowledge_stack_attachment , knowledge_stack_query_results, real_time_data_sources, delve_parent_chat_id, delve_info, prompt_response_metrics
chat_session_folders	채팅 중 세션 임시 저장 폴더	id, title , sort_order, created_at , is_expanded, extras
chat_sessions	채팅 세션 정보	id, title , created_at , archived_at, folder_id, splits_order, delve_info, is_vapor
chats	채팅방 정보 기록	id, title , model_vendor, model_name, config, parent_chat_id, created_at , chat_session_id , extras, delve_parent_chat_id, delve_info, prompt_response_metrics
recent_messages	최근 메시지	id, text, type, created_at
workspaces	사용자 지정 워크스페이스	id, name , location, extras, created_at

%APPDATA%\Msty\logs\app.log. 애플리케이션을 끄고 켜는 동작 등 프로세스 동작과 관련한 내용을 모두 기록하고 있다. 로그 생성 시각은 Unix Timestamp 형식으로 기록되며, 설치된 컴퓨터에 설정된 시간대를 따른다. 이 외에도 [그림 7]과같이, 호스트명(hostname)이 기록되므로 이 애플리케이션 및 대화를 생성한 것은 누구인지 식별이 가능하다.

```
{"level":30,"time":1730970800085,"pid":15380,"hostname":"Junseok","msg":"Running migrations..."}
```

[그림 7] app.log에 기록되는 로그 예시

3) Jan

JAN 애플리케이션의 경우, 아티팩트 폴더 하위 경로상에서 모든 데이터가 암호화되지 않은 채 저장되고 있다. 포렌식적으로 유의미한 데이터는 <표 7>의 구조와 같이 저장되는 것으로 분석되었다. 아티팩트에 기록되는 모든 시간대는 설치된 컴퓨터에 설정된 시간대를 따른다.

- 아티팩트 기록 경로: %APPDATA%\Jan\

<표 7> Jan의 아티팩트 세부 경로 및 항목

아티팩트 세부 경로	아티팩트 항목
%APPDATA%\Jan\Settings.json	<u>conversation data saving path</u>
%APPDATA%\Jan\data\logs\main.log	<u>program running info log</u> , GGUF file info
%APPDATA%\Jan\data\threads\jan_{ac count_created_time(Unix Timestamp)}\messages.jsonl	conversation ID, thread ID, <u>creation time(Unix Timestamp)</u> , <u>conversation log</u> , <u>upload file info</u> , error log
%APPDATA%\Jan\data\threads\jan_{ac count_created_time(Unix Timestamp)}\thread.json	<u>conversation log</u> , <u>creation time(Unix Timestamp)</u>
%APPDATA%\Jan\data\settings\@janh q\inference-openai-extension\settings.js on	<u>OpenAI API key info</u>

%APPDATA%\Jan\Settings.json. 대화 관련 데이터를 어디에 저장할지와 관련하여 평문으로 경로가 저장되어 있다. 실질적인 데이터 저장 경로는 “data_folder”로, 또는 사용자가 따로 경로를 지정할 것인지를 묻는 “quick_ask” 기능 on/off로 구성되어 있다. “data_folder”와 “quick_ask”의 기본 설정은 각각 “%APPDATA%\Jan\data”와 “false”이다.

%APPDATA%\Jan\config.json. 현재 Jan 애플리케이션의 버전과 사용자 상호작용 window의 정보가 기재되어 있다. 각각 “migrated_version”, “windowBounds” 항목으로 지정되어 있으며, 실시간으로 기록된다.

%APPDATA%\Jan\data\assistants\jan\assistant.json. 사용자의 Assistant로서 활동하는 Chatbot의 행동 지침 및 관련 정보가 담겨 있는 파일이다. 대화 목록을 새롭게 생성될 때마다 “id”와 “created_at” 정보가 각각 새롭게 생성되며, “created_at”에 기록되는 시간 정보는 Unix Timestamp 형태로 기록되며, 설치된 컴퓨터에 설정된 시간대를 따른다.

%APPDATA%\Jan\data\logs\main.log. 프로그램 실행 및 업데이트 시각, 모델 업로드 정보와 같은 프로그램 로그를 기록하는 파일이다. 구조는 [그림 8]과 같다. 세부 로그는 각각 [SPECS], [CORTEX], [APP] 로그로 기록되고 있다. [SPECS]는 업데이트 대상 PC 및 사용자의 기기 정보, [CORTEX]는 model 업로드 및 오류 정보를, [APP]은 애플리케이션 구동 정보를 기록하고 있다. 로그 생성 시각의 경우, 설치된 컴퓨터에 설정된 시간대를 따른다.

2024-11-04T07:14:55.648Z [SPECS]:: {information}

[그림 8] main.log에 기록되는 로그 구조

%APPDATA%\Jan\data\threads\jan_{account_created_time(Unix Timestamp)}\messages.jsonl. thread.json 파일을 작성하기 전, 사용자의 id, 역할, 상태, 콘텐츠 정보, 예러 정보 등을 기록하고 있는 파일이다. 세부 구조는 [그림 9]와 같다. 대화 생성 시각을 나타내는 “created”와 “updated”의 경우, 각각 Unix Timestamp 형식을 따르며, 설치된 컴퓨터에 설정된 시간대를 따른다. Chatbot(Jan)과 사용자 간의 대화가 각각 세분되어 한 줄씩 표기되고 있다.

```
{
  "id": "{account_created_time(Unix Timestamp))",
  "thread_id": "jan_{account_created_time(Unix Timestamp))",
  "type": "Thread",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": {
        "value": "{대화문}",
        "annotations": []
      }
    }
  ],
  "status": "ready",
  "created": 1730705935303,
  "updated": 1730705935303,
  "object": "thread.message"
}
```

[그림 9] messages.jsonl 파일에 기록되는 로그 구조

%APPDATA%\Jan\data\threads\jan_{account_created_time(Unix Timestamp))\thread.json. 사용자와 Chatbot 사이의 대화만을 기록한 파일이다. Chatbot ID, 제목, Chatbot 정보, 사용 중인 모델 정보, 파라미터, 생성 시각, 업데이트 시각, 마지막 대화 메시지가 기록되어 있다. 모든 로그 생성 시각은 Unix Timestamp 형식을 따르며, 설치된 컴퓨터에 설정된 시간대를 따른다.

%APPDATA%\Jan\data\settings\@janhq\inference-openai-extension\settings.json. 사용자가 기존에 입력한 OpenAI API 키값을 기록하고 있는 파일이다. “ControllerProps” 하위 “value” 항목에 그대로 기록이 되어 있다. 모든 로그 생성 시각은 Unix Timestamp 형식을 따르며, 설치된 컴퓨터에 설정된 시간대를 따른다.

4. 로컬 기반 애플리케이션 아티팩트 분석

로컬 기반 애플리케이션은 외부 API를 연결하거나, 외부 서비스로 제공되는 모델을 사용하지 않는다. 연구하는 과정에서 이러한 유형의 애플리케이션이 식별 되었으므로, 그 분석 결과를 본 장에 기술한다. 로컬 기반 애플리케이션은 gguf 형식의 이미 구축된 모델 파일을 로드하여 사용하므로, 민감한 정보 유출의 문제는 없다. 하지만, 가짜 뉴스, 스팸 메일 생성 등 악의적인 콘텐츠를 생성하는 데 여전히 악용될 수 있다.

1) LM Studio

포렌식적으로 유의미한 데이터는 <표 8>과같이 저장되는 것으로 분석되었다. 아티팩트에 기록되는 모든 시간대는 설치된 컴퓨터에 설정된 시간대를 따른다.

- 아티팩트 기록 경로:
 - %APPDATA%\LM Studio
 - %USERPROFILE%\cache\lm-studio

<표 8> LM Studio 아티팩트 세부 경로 및 항목

아티팩트 세부 경로	아티팩트 항목
%USERPROFILE%\cache\lm-studio\conversations.json	<u>chat conversations log</u> , model info, <u>uploaded file metadata</u>
%APPDATA%\LM Studio\logs\main.log	<u>app running log</u> , loaded model info, error log
%USERPROFILE%\cache\lm-studio\user-files	<u>copies of uploaded files</u>

%APPDATA%\LM Studio\config.json. 사용자가 상호작용 중인 GUI 창의 가로 및 세로 폭 등 설정 상태를 실시간으로 표기하는 파일이다.

%APPDATA%\LM Studio\settings.json. 사용자가 사용하고자 하는 모델 gguf 파일의 다운로드 경로 및 언어, 구성, 채팅, 개발자 모드, UI를 설정하는 파일이다.

%USERPROFILE%\cache\lm-studio\conversations.json. 사용자가 챗봇과 함께 나눈 대화를 JSON 형식으로 기록한 파일이다. 각 대화문을 thread 단위로 나눈 뒤 각각에 소요된 토큰 등 자원 소요 현황을 기록하고 있다. 각 대화의 생성 시각은 모두 Unix Timestamp 형식을 따르고 있으며, 설치된 컴퓨터에 설정된 시간대를 따른다. 마지막으로 사용한 로컬 gguf file의 정보는 “lastUsed Model” 항목에 표시되고 있다. 특히 파일을 첨부하였을 경우, 해당 파일의 meta data는 <표 9>와 같은 구조로 표시된다.

<표 9> conversations.json 파일에 기록된 첨부파일 metadata 구조

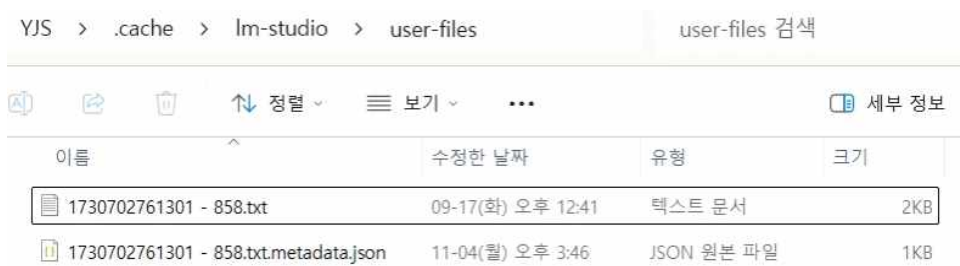
항목	구조
type	file
fileIdentifier	{created_time(Unix_TimeStamp)} - {random Number}.{extension_name}
fileType	text 파일인 경우 - text/plain
sizeBytes	업로드된 파일의 크기(bytes)

%APPDATA%\LM Studio\logs\main.log. 애플리케이션 구동 방식과 로드된 모델의 정보 등 기본적인 프로그램 실행 로그 기록되는 파일이다. 애플리케이션 업데이트, 설치, 시작, 모델 업로드 시각은 설치된 컴퓨터에 설정된 시간대를 따른다. 로그 구조는 [그림 10]과 같다.

[2024-11-04 15:30:01.464] [info] {log information}

[그림 10] LM Studio의 main.log에 기록되는 로그 구조

%USERPROFILE%\cache\lm-studio\user-files. 사용자가 챗봇과의 대화 도중 업로드한 파일들을 모두 복사하여 저장하는 폴더이다. 복사된 파일의 이름은 앞서 설명한 %USERPROFILE%\cache\lm-studio\conversations.json의 file Identifier에 명시된 것과 같은 이름으로 저장되며, 파일의 메타데이터는 따로 {created_time(Unix_TimeStamp)} - {random Number}.{extension_name}.metadata.json 파일로 복사본과 함께 저장된다. [그림 11]은 저장된 파일의 내용이 담겨있는 user-files 폴더의 캡처 사진이다.



[그림 11] user-files 폴더에 저장된 업로드 파일의 복사본

2) Sanctum

포렌식적으로 유의미한 데이터는 <표 10>의 구조와 같이 저장되는 것으로 분석되었다. 아티팩트에 기록되는 모든 시간대는 설치된 컴퓨터에 설정된 시간대를 따른다.

- 아티팩트 기록 경로:
 - %LOCALAPPDATA%\ai.sanctum.chat\
 - %APPDATA%\ai.sanctum.chat\

<표 10> Sanctum 아티팩트 세부 경로 및 항목

아티팩트 세부 경로	아티팩트 항목
%APPDATA%\ai.sanctum.chat\accounts-v2\{account_id}\settings-v2.json	<u>user account info</u> , local model gguf file info
%LOCALAPPDATA%\ai.sanctum.chat\logs\rust.log	<u>conversation log</u> , <u>uploaded file original path</u> , <u>app running log</u> , error log

%APPDATA%\ai.sanctum.chat\settings-v2.json. 로컬 모델 파일, 캐시 저장 경로를 설정하고, 사용한 모델에 관한 정보를 기록하고 있다. 특히 “last_active_account_id”가 기록되어 있어, 마지막으로 애플리케이션을 구동한 사람이 누구인지 분석할 수 있다.

%APPDATA%\ai.sanctum.chat\accounts-v2\{account_id}\settings-v2.json. 대규모 언어 모델 사용자의 계정 정보 및 사용 중인 로컬 gguf 파일의 정보에 대해 기록하고 있는 파일이다.

%LOCALAPPDATA%\ai.sanctum.chat\logs\rust.log. 사용자와 ai 사이에 나눈 대화 내용 및 첨부한 파일 경로, 앱 구동 기록을 기록하고 있는 통합 로그 파일이다. 로그 생성 시각은 설치된 컴퓨터에 설정된 시간대를 따른다. 특히 사용자가 파일을 업로드할 경우, 해당 파일의 원본 경로를 로그에 기록한다는 특징이 있다. 다만, 해당 파일을 따로 복사하여 저장하지는 않는다. [그림 12]는 첨부파일 관련 rust.log 기록이다.

```
src-tauri\src\database\schemas\message.rs:302: [34mDEBUG[0m: add_chat_message_id: documents=["{%USERPROFILE%}\\Desktop\\lets_recover.py"] - 1s
```

[그림 12] rust.log에 기록된 첨부파일 관련 로그 예시

5. 결과 및 논의

1) 분석 결과 요약

본 페이지에서는 현재까지 분석한 Monica, Msty, Jan, LM Studio, Sanctum 총 5가지 대규모 언어 모델 애플리케이션들을 디지털 포렌식 관점에서 유의미한 결과 위주로 요약한다. 민감한 데이터의 외부 유출 여부를 포함한 사용자의 주 행위를 밝힐 수 있는 흔적인 사용자의 애플리케이션 사용 흔적, 채팅 기록, 업로드한 정보의 메타데이터 등이 있다. 외부 API 사용, 채팅 기록, 업로드 파일 관련 기록, 애플리케이션 기록을 위주로 8가지의 세부 기준을 정하고, 분석 결과를 <표 11>과 같이 정리하였다.

<표 11> 대화형 AI 애플리케이션 아티팩트 분석 결과 요약

애플리케이션 유의미한 정보	Monica	Msty	Jan	LM Studio	Sanctum
프로그램 로그	✓	✓	✓	✓	✓
API 연결 지원 여부		✓	✓		
API 키 정보 기록		✓	✓		
로컬 모델 지원 여부	✓	✓	✓	✓	✓

채팅 기록	▲	✓	✓	✓	✓
파일 업로드 가능	✓	✓		✓	✓
업로드한 파일의 로그	▲	✓		✓	✓
업로드한 파일 복사본 확인 가능	▲	✓		✓	

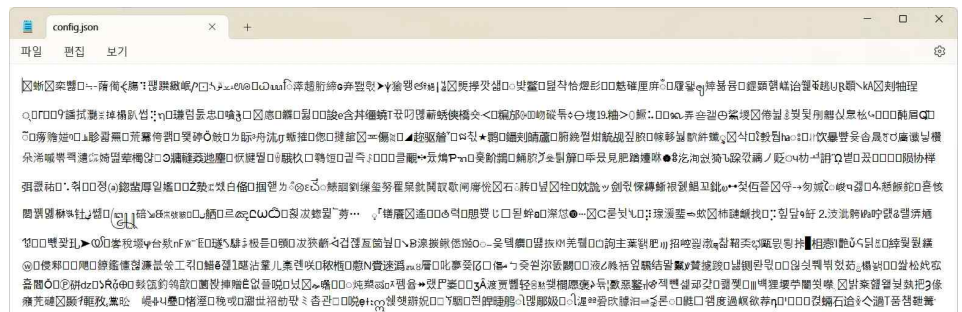
(✓: 아티팩트 확인 가능 / ▲: 상황에 따라 아티팩트 확인 여부 달라짐)

프로그램 로그는 모든 애플리케이션이 공통으로 남기고 있는 아티팩트로 분석되었다. API 연결 지원의 경우, Monica, LM Studio, Sanctum을 제외한 모든 애플리케이션에서 지원하였다. 다만, Monica는 사용자 지정 API 키 입력이 불가능하지만, 외부 대화형 AI 서비스(ChatGPT 등)와의 연결이 가능하다는 특징이 있었다. API 연결이 가능한 모델의 경우 모두 API 키 정보를 아티팩트에 기록하고 있었다. 로컬 모델의 경우 모든 애플리케이션에서 지원하였다. 채팅 기록의 경우 Monica를 제외한 모든 애플리케이션에서 아티팩트를 남기고 있는 것이 확인되었다. Monica 애플리케이션은 외부 대화형 AI 서비스를 사용할 경우 관련 채팅 기록을 남기지 않고 있다. 파일 업로드 기록의 경우 Jan을 제외한 모든 애플리케이션에서 지원하였다. 다만, Monica는 외부 대화형 AI 서비스를 사용할 경우 파일 관련 아티팩트가 전혀 남지 않았고, Monica 모델을 사용하면 클라우드에 저장된 업로드 파일 URL을 남겼다. 또한 Sanctum은 파일 업로드가 가능하고, 업로드 파일의 로그 확인이 가능하지만 파일 원본 경로만 확인될 뿐 업로드 파일 복사본은 확인할 수 없었다.

2) 분석의 한계점: Monica

Monica 애플리케이션의 경우 <표 11>에서 명시한 것과 같이, 외부 대규모 언어 모델 서비스와 연결하여 챗봇과 대화하는 것이 가능하나 사용자가 API 키를 임의로 지정하는 기능이 존재하지 않기 때문에 사용자의 API 키는 기록이 남지 않는다. 외부 대규모 언어 모델 애플리케이션을 활용한 대화의 경우 ChatLRUCache 값의 변화만 기록될 뿐, 어떠한 상세 로그도 남지 않았다. 또한 %APPDATA%\Monica\config.json은 [그림 13]과같이 기록된 데이터를 확인할 수 없도록 암호화되어 있다. 이는 개발자가 애플리케이션 개인정보의 보안을 위

해 민감한 데이터를 보호하기 위해서 암호화한 것으로 생각된다. 이에 Monica 애플리케이션의 경우 로컬 대규모 언어 모델 모델을 사용하는 특수한 경우를 제외하고는 디지털 포렌식 관점에서 유의미한 아티팩트를 확인하는 데 한계가 있었다.



[그림 13] 사용자가 읽기 어렵도록 암호화되어 있는 Monica의 config.json

IV. 대화형 AI 데이터의 디지털 포렌식 활용 방안

이 장에서는 Msty 대화형 AI 애플리케이션 아티팩트를 활용한 기업의 민감 정보에 대한 보안사고 조사 시나리오를 통해, 대화형 AI 데이터의 디지털 포렌식 활용 방안을 제시한다.

1. 기업 보안사고 조사 가상 시나리오 개요

모 기업 디지털 기기 정기 보안점검 중 허가되지 않은 대화형 AI 애플리케이션을 사용한 흔적이 발견되어, 디지털 포렌식 수사 과정에서 입수한 “김직원”의 PC를 분석하는 상황을 가정한다. 기업 감사팀은 “김직원”이 업무용 PC에서 허가되지 않은 대화형 AI 애플리케이션을 사용하고, 기밀 자료를 외부에 업로드한 보안 사고의 흔적을 찾는다.

기업 감사팀은 %APPDATA%\Msty\ 폴더를 파일 탐색기로 진입한 뒤, 앞에서 확인한 Msty 애플리케이션 아티팩트들을 모두 분석하였다.

2. 아티팩트 분석

%APPDATA%\Msty\logs\app.log 분석. 해당 파일을 분석한 결과, [그림 14]와 같은 기록이 남아있는 것을 발견하였다. 이때 로그 생성 시각은 Unix Timestamp 형식이므로, 사용자가 알아보기 쉬운 시각 형태로 변환한 뒤 이벤트 순서를 분석하였다.

- 최근 애플리케이션을 켜 시각: **2024-11-29 19:24:31 (KST)**
- ChatGPT 4o 모델을 활용하여 대화를 시작한 시각:
2024-11-29 19:31:01 (KST)
- 대화를 마치고 앱을 종료한 시각: **2024-11-29 19:36:15 (KST)**

```
{ "level":30,"time":1730968102192,"pid":2120,"hostname":"junseok","msg":
:"Running migrations..." }
...
{ "level":30,"time":1732875871597,"pid":1476,"hostname":"junseok","msg":
:"Running migrations..." }
...
{ "level":30,"time":1732875871895,"pid":1476,"hostname":"junseok","msg":
:"App ready." }
...
{ "level":30,"time":1732876550268,"pid":1476,"hostname":"junseok","msg":
:"Conversation started with openai using gpt-4o and options
{ \"maxTokens\":1000,\"temperature\":1,\"topP\":1,\"frequencyPenalty\":0,\"
presencePenalty\":0}" }
{ "level":30,"time":1732876555458,"pid":1476,"hostname":"junseok","msg":
:"Estimating cost for model gpt-4o with 477 input tokens and 377
output tokens" }
...
{ "level":30,"time":1732876575716,"pid":1476,"hostname":"junseok","msg":
:"Killing MstyLocal AI service pid: 17668..." }
```

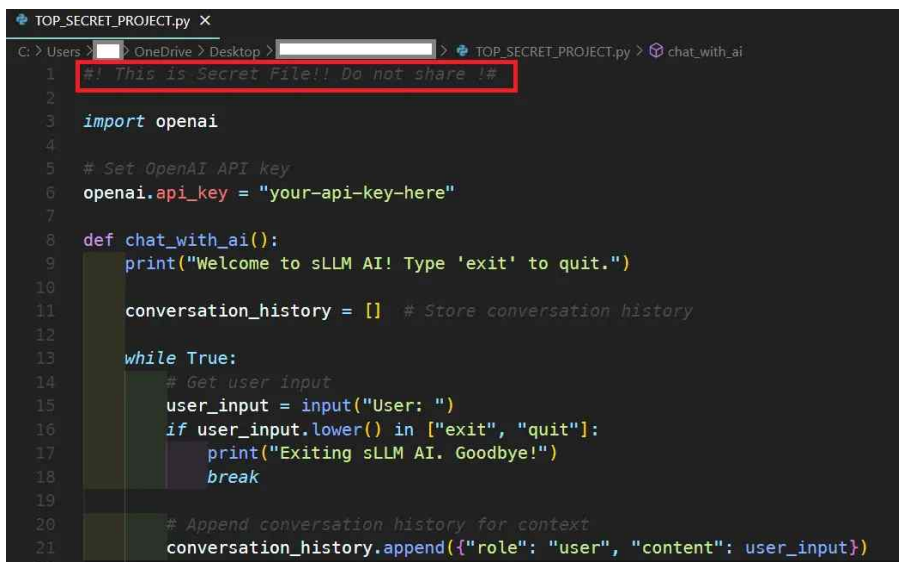
[그림 14] app.log 세부 로그

%APPDATA%\Msty\msty.db 분석. 해당 파일의 “chats” 및 “chat_messages” 테이블을 통해서 대화와 관련된 흔적을 발견할 수 있다. 파일 업로드와 관련된 대화는 2024-11-29 19:35:50 (KST)에 이뤄졌고, 상세 로그는 [그림 15]와 같다.

```
{
  "images": [],
  "documents": [
    {
      "size": 1224,
      "type": "text/x-python",
      "path": "%APPDATA%\attachments\\1732876539441-TOP_SECRET_PROJECT.py",
      "name": "1732876539441-TOP_SECRET_PROJECT.py",
      "org_path": "%USERPROFILE%\OneDrive\Desktop\\{censored_directory}\\TOP_SECRET_PROJECT.py",
      "org_name": "TOP_SECRET_PROJECT.py",
      "folder": "attachments"
    }
  ],
  "youtube_links": []
}
```

[그림 15] msty.db에 기록된 업로드 관련 로그 (attachments 칼럼)

%APPDATA%\Msty\attachments 분석. 해당 폴더에는 업로드한 파일에 대한 복사본이 저장된다. [그림 16]과같이 복사본을 통해서 실제로 보안 사고가 발생한 파일의 내용을 확인할 수 있다. 또한, [그림 17]과같이 기밀 파일과 복사된 업로드 파일의 해시값을 비교하여 서로 동일한지 확인하였고, 실제로 동일함을 확인하였다.



```
TOP_SECRET_PROJECT.py X
C: > Users > [redacted] > OneDrive > Desktop > [redacted] > TOP_SECRET_PROJECT.py > chat_with_ai
1  #! This is Secret File!! Do not share !#
2
3  import openai
4
5  # Set OpenAI API key
6  openai.api_key = "your-api-key-here"
7
8  def chat_with_ai():
9      print("Welcome to sLLM AI! Type 'exit' to quit.")
10
11     conversation_history = [] # Store conversation history
12
13     while True:
14         # Get user input
15         user_input = input("User: ")
16         if user_input.lower() in ["exit", "quit"]:
17             print("Exiting sLLM AI. Goodbye!")
18             break
19
20         # Append conversation history for context
21         conversation_history.append({"role": "user", "content": user_input})
22
```

[그림 16] 업로드 파일의 복사본

이름	TOP_SECRET_PROJECT.py
크기	1268 바이트 (1 KiB)
CRC32	C50A4D24
CRC64	A8B7F049FC0C874F
SHA256	FA9D4031A7F49CC8AF97A7139B459C239EEE6A6DE4FD4398427A5F5D262D9ECD
SHA1	251854EBA2E7B1635581EF5A04D1EBF19704E92F
BLAKE2sp	D0690D93BD8885684AF523E574C586B46833067ACC602C15596A753042DF09AB
이름	1732876539441-TOP_SECRET_PROJECT.py
크기	1268 바이트 (1 KiB)
CRC32	C50A4D24
CRC64	A8B7F049FC0C874F
SHA256	FA9D4031A7F49CC8AF97A7139B459C239EEE6A6DE4FD4398427A5F5D262D9ECD
SHA1	251854EBA2E7B1635581EF5A04D1EBF19704E92F
BLAKE2sp	D0690D93BD8885684AF523E574C586B46833067ACC602C15596A753042DF09AB

[그림 17] 기밀 파일 및 복사본의 해시값 비교

3. 분석 결과 종합

앞서 확인한 사실을 바탕으로, “김직원”은 업무용 PC에 허가되지 않은 대화형 AI 애플리케이션(Msty)을 다운로드 및 설치하였다는 점, 사전에 발급받은 OpenAI API 키를 이용하여 Msty 내부 ChatGPT 챗봇과 대화를 하였다는 점, 그리고 기업 기밀 자료를 ChatGPT 챗봇에 업로드하였다는 점을 확인할 수 있었다. 따라서 최종적으로 “김직원”에 의해 민감한 정보가 외부에 유출되는 보안 사고가 일어났음을 확인할 수 있었다. “김직원”이 챗봇을 사용한 기록을 바탕으로 대화 내용을 복원하면 [그림 18]과 같고, 시나리오에 대한 타임라인(Timeline)은 <표 12>와 같다.

<표 12> “김직원”에 의한 기업 내 보안사고 타임라인

주요 시각	행위
2024-11-07 17:28:22 (KST)	Msty를 최초로 PC에 설치
2024-11-29 19:24:31 (KST)	보안 정기 점검 전, 가장 최근에 Msty 시작
2024-11-29 19:31:01 (KST)	“김직원”이 지정 OpenAI API Key를 이용하여 ChatGPT 3.5 turbo 및 4o 모델과 대화 시작
2024-11-29 19:35:50 (KST)	“김직원”이 기업 기밀 자료를 외부 ChatGPT 4o 모델에 업로드
2024-11-29 19:36:15 (KST)	“김직원”이 모든 대화를 마치고 Msty 종료



[그림 18] 챗봇 대화 재구성

V. 결론 및 향후 연구

최근 대규모 언어 모델 기반 AI 서비스의 급성장으로 사용자의 개인정보 및 기관의 기밀 자료가 AI 학습에 활용되는 문제가 제기되고 있다. 특히, 대화형 AI를 활용한 업무 효율화 과정에서 기밀 자료 유출 가능성이 우려되며, 가짜 댓글이나 피싱 등 악의적 콘텐츠 생성에 연루된 대화형 AI 애플리케이션에 대한

포렌식 분석이 필요하다.

본 논문에서는 대표적으로 많이 사용되는 데스크톱 버전 대규모 언어 모델 애플리케이션인 Monica, Msty, Jan, LM Studio, Sanctum을 대상으로 분석을 진행하였다. 애플리케이션마다 아티팩트가 어떤 형태로 저장되고 있는지 분석하였으며, 보안 사고로 인한 예상되는 민감한 정보가 유출되는 가상의 시나리오를 통해 본 연구 결과가 유용함을 보였다.

5개의 애플리케이션 중 Monica는 암호화가 적용되어 사용자의 흔적 확인이 불가능했다. 이 경우를 통해서 암호화된 대화형 AI 애플리케이션의 존재를 확인할 수 있었다. 향후 연구로는 Monica와 같이 암호화를 적용해서 아티팩트를 저장하는 애플리케이션을 대상으로 암호화 알고리즘 분석 및 복호화를 통한 아티팩트 확보를 제안한다.

마지막으로 본 연구 결과가 악의적으로 콘텐츠를 생성하는 악용자의 컴퓨터를 조사하는 데 활용되고, 민감한 정보 유출을 미연에 방지하는 데 기여할 수 있기를 기대한다.

참고문헌

- Cho, K., Park, Y., Kim, J., Kim, B., & Jeong, D. (2025). Conversational AI forensics: A case study on ChatGPT, Gemini, Copilot, and Claude. *Forensic Science International: Digital Investigation*, 52, 301855. doi:10.1016/j.fsidi.2024.301855.
- Walker, C., Gharaibeh, T., Alsmadi, R., Hall, C., & Baggili, I. (2024.07.). Forensic Analysis of Artifacts from Microsoft's Multi-Agent LLM Platform AutoGen. In *Proceedings of the 19th International Conference on Availability, Reliability and Security* (pp. 1-9). doi:10.1145/3664476.3670908.
- Dragonas, E., Lambrinoudakis, C., & Nakoutis, P. (2024). Forensic analysis of OpenAI's ChatGPT mobile application. *Forensic Science International: Digital Investigation*, 50, 301801. doi:10.1016/j.fsidi.2024.301801.

Abstract

Forensic Artifact Analysis of Conversational AI Applications on Windows OS

Yeo, Jun-Seok* · Kim, Chae-Min · Kim, Dong-Hyun**

This study is This study aims to analyze digital forensic artifacts of five Windows-based conversational AI applications to address growing concerns over privacy breaches and data misuse in generative AI systems. We conduct a systematic investigation and categorization of artifacts produced through user interactions with these applications, employing forensic analysis techniques. The findings of this study can be effectively utilized in digital forensic investigations through a simulated scenario of sensitive information leakage. Our findings contribute to advancing the investigation of malicious content generation as well as the development of proactive measures for ensuring data security in AI applications.

Key words: Digital forensics, application forensics, artifact analysis, conversational AI

* Undergraduate Student, Dept. of Computer Science, Korea Military Academy

** Assistant Professor, Dept. of Computer Science, Korea Military Academy

원고투고일: 2024년 12월 01일

심사완료일: 2024년 12월 17일

원고수정일: 2024년 12월 21일

게재확정일: 2024년 12월 27일